

Unit:3 Apache Hadoop and HDFS

1. History of Hadoop
2. Apache Hadoop
3. Analyzing Data with Unix tools
4. Analyzing Data with Hadoop
5. Hadoop Streaming
6. Hadoop Ecosystem
7. The Design of HDFS
8. HDFS Concepts
9. Command Line Interface
10. Hadoop file system interfaces
11. Data flow, Data Ingest with Flume and Scoop and Hadoop archives
12. Hadoop I/O: Compression
13. Serialization Avro and File-Based Data structures

Hadoop Introduction

Hadoop is an open-source framework that enables the distributed processing of large datasets across clusters of computers using simple programming models. Its development was driven by the need to process vast amounts of data efficiently, and it has become a cornerstone of big data processing.

Hadoop: An Open-Source Framework

Hadoop is an open-source framework overseen by the Apache Software Foundation. Written primarily in Java, it enables the storage and processing of vast datasets using a cluster of commodity hardware. The core challenge with big data are:

- Storing large amounts of data.
 - Processing that data efficiently.
- Traditional methods, such as RDBMS, are insufficient due to the heterogeneity of big data. Hadoop addresses these issues by providing scalable and cost-effective solutions for storing and processing huge datasets.

Hadoop core components include:

- **Hadoop Distributed File System (HDFS):** For distributed storage.
- **Yet Another Resource Negotiator (YARN):** For managing resources and job scheduling.
- **MapReduce:** For distributed data processing using the divide-and-conquer approach.
- **Hadoop Common:** Provides shared libraries, utilities, and tools needed by Hadoop and its ecosystem.

History of Hadoop

- **2002:**
Doug Cutting and Mike Cafarella began work on the **Apache Nutch** project, aiming to build a web search engine capable of indexing 1 billion pages. However, the cost of infrastructure—about **\$500,000 in hardware** and **\$30,000 monthly in running costs**—made this goal impractical.
- **2003:**
While searching for scalable solutions, they found Google's research paper on the **Google File System (GFS)**. It offered an efficient way to store large datasets across multiple machines, which solved Nutch's storage issue—but not processing.
- **2004:**
Google released another paper introducing **MapReduce**, a programming model for processing massive datasets in parallel. This completed the puzzle for Nutch: GFS for storage and MapReduce for processing.
- **2005:**
In 2005, Doug Cutting and Mike Cafarella were working on the Nutch search engine and realized they needed a better way to store and process huge amounts of data. After reading Google's papers on GFS and MapReduce, they started implementing similar ideas inside Nutch. This effort led to the creation of **Hadoop**, which he named after his son's toy elephant.

History of Hadoop

- **2006:** Doug Cutting joined Yahoo! and worked on scaling Nutch. At Yahoo, he separated its distributed computing components, leading to the creation of **Hadoop**, named after his son's toy elephant. Hadoop was designed to operate across thousands of nodes, integrating **GFS (Google File System)** and **MapReduce**.
- **2007:** Yahoo! successfully tested **Hadoop** on a **1,000-node cluster**, which validated the scalability of the system.
- **2008:** In **January**, Yahoo! released **Hadoop** as an open-source project under the **Apache Software Foundation (ASF)**. By **July 2008**, Hadoop was successfully tested on a **4,000-node cluster**.
- **2009:** Hadoop successfully demonstrated its ability to process **1 petabyte (PB)** of data in **less than 17 hours**, handling billions of searches and indexing millions of web pages. Doug Cutting left Yahoo! and joined **Cloudera**, aiming to expand Hadoop's use across various industries.
- **2011:** In 2011, the **Apache Software Foundation released Hadoop version 1.0**, marking its first stable release and making it a fully supported and production-ready platform for big data processing.

History of Hadoop

- **2013:** Hadoop **version 2.0.6** was released in 2013, introducing major improvements such as **YARN (Yet Another Resource Negotiator)**, which allowed better resource management and support for multiple processing frameworks beyond MapReduce.
- **2017:** The **major release of Hadoop 3.0** came in December 2017, featuring enhanced scalability, improved storage management, and new enterprise-ready features, further solidifying Hadoop as a leading platform for large-scale data processing.

Apache Hadoop

Apache Hadoop is an open-source framework that facilitates the storage and processing of vast datasets in a distributed computing environment. It addresses the challenges associated with managing large volumes of data by leveraging a network of computers, known as a cluster, to share the workload.

Key Features

1. Distributed Computing:

Hadoop operates on a cluster of machines, which allows it to divide large data sets into smaller chunks. Each chunk is processed in parallel on different nodes, significantly speeding up data processing times.

2. Scalability:

One of Hadoop's standout features is its ability to scale horizontally. Organizations can easily add more nodes to the cluster as data volumes grow, without needing to overhaul the existing infrastructure. This makes it a flexible solution for businesses of all sizes.

3. Fault Tolerance:

Hadoop is designed with resilience in mind. It automatically replicates data across multiple nodes, ensuring that if one node fails, the data remains accessible from another node. This replication mechanism helps maintain data integrity and availability.

Apache Hadoop

4. Handling Diverse Data Types:

Unlike traditional databases that primarily manage structured data, Hadoop can efficiently store and process both structured and unstructured data. This includes text files, images, videos, and more, allowing organizations to gain insights from a wide array of data sources.

5. Cost-Effectiveness:

4. Hadoop runs on commodity hardware, which means organizations can use inexpensive, off-the-shelf servers instead of costly proprietary systems. This significantly reduces the overall cost of data storage and processing.

Core Components

1. Hadoop Distributed File System (HDFS):

HDFS is the storage layer of Hadoop, designed to handle large files by splitting them into blocks and distributing them across the cluster. It ensures high throughput and fault tolerance.

2. MapReduce:

This is the processing layer of Hadoop, which enables the execution of distributed computations. It consists of two main functions: the "Map" function, which processes input data into key-value pairs, and the "Reduce" function, which aggregates results from the Map phase.

3. YARN (Yet Another Resource Negotiator):

YARN is responsible for resource management and job scheduling within the Hadoop ecosystem. It allows multiple processing frameworks to run on the same cluster, enhancing flexibility.

4. Hadoop Common:

Hadoop Common provides the essential libraries, utilities, and tools required by all Hadoop modules. It offers the basic functionality needed to support HDFS, MapReduce, and YARN, such as Java libraries, file system interfaces, and configuration management.

Analyzing Data with Unix tools

- **Unix tools** are small command-line programs used in Unix-like systems (Linux, macOS) to work with files, text, and processes. Each tool does one simple task like searching text, sorting data, counting words, or combining files. When you join these small tools together using pipes (`|`), they become very powerful for analyzing and processing data. They are fast, lightweight, and widely used in data engineering, big data, and system administration.
- UNIX tools are a collection of small, powerful command-line programs available on UNIX-based operating systems (Linux, macOS, BSD). They follow the philosophy:
- **“Do one thing, and do it well.”**
- Each tool performs a specific function such as:
 - Viewing data
 - Searching text
 - Filtering lines
 - Sorting data
 - Counting words
 - Transforming text
 - Comparing files
 - Automating tasks
- Individually these tools are simple, but when combined, they become extremely powerful for **data processing**, especially when the dataset is large.

Analyzing Data with Unix tools

Why Unix Tools for Big Data?

Before the emergence of distributed computing frameworks like Hadoop, software engineers commonly used Unix tools to process large volumes of data. These tools provide efficient, reliable, and flexible ways to manipulate text and structured data. The main reasons for their continued relevance in big data contexts are:

- **High Performance:** Unix tools such as `grep`, `awk`, `sed`, and `sort` are lightweight and optimized for processing data streams directly, allowing fast execution even on large files.
- **Mature and Reliable:** Developed and refined over decades, Unix tools are robust and well-tested, making them dependable for production-level data processing tasks.
- **Pipeline-Oriented Design:** commands can be linked using pipes (`|`), enabling modular and readable workflows for sequential data transformations.
- **Ideal for Moderate Datasets:** While not designed for petabyte-scale processing, Unix tools efficiently handle datasets from a few gigabytes to tens of gigabytes, making them suitable for preprocessing, exploratory analysis, and intermediate transformations.

In summary, Unix tools combine speed, stability, and simplicity, providing a strong foundation for data analysis and serving as a stepping stone toward distributed frameworks like Hadoop.

Analyzing Data with Unix tools

1. Analyzing data with Unix tools can be highly efficient due to their lightweight and command-line interface. Here's a deeper look into some common tools and how to use them effectively:

1. `grep`:

- **Purpose:** Search for specific patterns within files.
- **Usage:**

```
bash
```

```
grep 'pattern' filename
```

- **Example:** Find all lines containing "error" in a log file.

```
bash
```

```
grep 'error' logfile.txt
```

2. `awk`:

- **Purpose:** Text processing and data extraction.
- **Usage:**

```
bash
```

```
awk 'pattern { action }' filename
```

- **Example:** Print the first and second columns of a CSV file.

```
bash
```

```
awk -F',' '{print $1, $2}' data.csv
```

Analyzing Data with Unix tools

3. sed :

- **Purpose:** Stream editing for filtering and transforming text.
- **Usage:**

```
bash
sed 's/old/new/g' filename
```

- **Example:** Replace "foo" with "bar" in a text file.

```
bash
sed 's/foo/bar/g' file.txt
```

4. sort :

- **Purpose:** Sort lines of text files.
- **Usage:**

```
bash
sort filename
```

- **Example:** Sort a file alphabetically.

```
bash
sort data.txt
```

5. uniq :

- **Purpose:** Report or omit repeated lines.
- **Usage:**

```
bash
uniq filename
```

- **Example:** Count unique lines in a sorted file.

```
bash
sort data.txt | uniq -c
```

Analyzing Data with Unix tools

6. `cut` :

- **Purpose:** Remove sections from each line of files.

- **Usage:**

```
bash
cut -d'delimiter' -fN filename
```

- **Example:** Extract the second column from a tab-separated file.

```
bash
cut -f2 data.tsv
```

7. `wc` :

- **Purpose:** Count lines, words, and characters in files.

- **Usage:**

```
bash
wc filename
```

- **Example:** Count the number of lines in a file.

```
bash
wc -l data.txt
```

8. `find` :

- **Purpose:** Search for files in a directory hierarchy.

- **Usage:**

```
bash
find path -name 'pattern'
```

- **Example:** Locate all text files in the current directory.

```
bash
find . -name '*.txt'
```

Analyzing Data with Unix tools (Example Workflow)

1. Filtering Data:

- Start by filtering relevant data from a log file:

```
bash
grep '2023' logs.txt > filtered_logs.txt
```

2. Processing Data:

- Use `awk` to extract specific columns and sort the results:

```
bash
awk '{print $2, $5}' filtered_logs.txt | sort | uniq -c > summary.txt
```

3. Analyzing Results:

- Count the number of lines in the summary for final insights:

```
bash
wc -l summary.txt
```

Analyzing Data with Hadoop

Hadoop is a distributed computing framework designed to process very large datasets across clusters of commodity hardware. Unlike traditional Unix tools that work efficiently on moderate-sized data, Hadoop can handle terabytes or petabytes of data reliably. The process of analyzing data in Hadoop typically involves the following aspects:

- **Distributed Storage:** Hadoop stores data in the Hadoop Distributed File System (HDFS), which splits large files into blocks and distributes them across multiple nodes. This ensures fault tolerance and high throughput for large-scale data access.
- **Parallel Processing with MapReduce:** Hadoop uses the **MapReduce programming model** to analyze data in parallel. The “Map” function processes input data into intermediate key-value pairs, and the “Reduce” function aggregates these results to produce the final output. This enables large datasets to be processed efficiently across many nodes.
- **Scalability:** Hadoop can scale horizontally by adding more nodes to the cluster. This makes it capable of handling ever-growing datasets without significant redesign of workflows.
- **Flexibility:** Hadoop can process structured, semi-structured, and unstructured data, making it suitable for diverse data analysis tasks, including log analysis, text mining, and machine learning preprocessing.
- **Ecosystem Support:** The Hadoop ecosystem includes tools like **Hive** (SQL-like querying), **Pig** (data flow scripts), and **HBase** (NoSQL storage), which simplify complex data analysis tasks.

Analyzing Data with Hadoop

Workflow for Analyzing Data with Hadoop

Analyzing data in Hadoop follows a step-by-step process that makes handling large datasets easier:

1. Collect Data:

Gather data from different sources like logs, databases, sensors, or social media. Hadoop can handle structured, semi-structured, and unstructured data.

2. Load Data into HDFS:

Store the data in **Hadoop Distributed File System (HDFS)**. HDFS breaks large files into smaller blocks and spreads them across multiple nodes for safe storage and faster access.

3. Preprocess Data:

Clean, filter, and prepare the data for analysis. You can use Unix tools, Hadoop MapReduce jobs, or tools like **Pig** and **Hive**.

Analyzing Data with Hadoop

4. Process Data with MapReduce:

- **Map phase:** Break data into key-value pairs and apply mapping functions.
- **Shuffle and Sort:** Group and sort intermediate results for the next step.
- **Reduce phase:** Combine and summarize the results to get the final output.

5. Store and Retrieve Output:

Save the processed results back in HDFS or other storage systems like **HBase** for further use.

6. Analyze and Visualize:

Use Hive, Pig, or visualization tools to explore the data and create reports or dashboards.

7. Refine Workflow:

Make improvements by optimizing MapReduce jobs, tuning the cluster, or adding more data to get better results.

comparison of analyzing data with Hadoop vs. analyzing data with Unix:

Aspect	Analyzing Data with Hadoop	Analyzing Data with Unix
Scale	Designed for big data (terabytes to petabytes).	Suitable for small to medium datasets.
Storage	Uses HDFS for distributed storage across many machines.	Uses local file systems (e.g., ext4, NTFS).
Processing Model	Parallel processing via MapReduce or Apache Spark.	Sequential or simple pipelined processing via shell scripts.
Tools & Ecosystem	Rich ecosystem: Hive, Pig, Spark, HBase, etc.	Command-line tools: awk, sed, grep, cut, etc.
Fault Tolerance	High fault tolerance; nodes can fail without losing data.	Low fault tolerance; failure of a single operation may cause loss.
Ease of Use	Requires knowledge of Hadoop stack; higher learning curve.	Simple to use for those familiar with shell scripting.
Performance	Optimized for large-scale batch processing.	Faster for small data but not scalable.
Infrastructure	Requires a cluster of machines; cloud or on-premise setup.	Runs on a single server or machine.
Use Case	Best for enterprise-level data analysis, big data processing.	Ideal for quick log parsing, small file manipulation.
Cost	Needs infrastructure setup; may involve cloud or cluster costs.	Minimal cost; runs on existing Unix/Linux systems.

Hadoop Streaming

Hadoop Streaming is a command-line utility(feature) in Apache Hadoop that allows developers to write MapReduce programs in any language capable of reading from standard input (STDIN) and writing to standard output (STDOUT). Unlike traditional Java-based MapReduce, Hadoop Streaming enables the use of scripting languages such as Python, Bash, Perl, Ruby, PHP, or R.

It simplifies the development of MapReduce programs by abstracting Java-specific APIs, allowing users to focus on data processing logic. Hadoop Streaming leverages Hadoop's distributed architecture to execute tasks in parallel, manage data distribution, and ensure fault tolerance, making it ideal for rapid prototyping, education, and experimental data processing.

Key Features of Hadoop Streaming

1. Language Independence

- Hadoop Streaming allows MapReduce programs to be written in any language capable of reading from STDIN and writing to STDOUT.
- This makes it accessible to students and professionals who may not be familiar with Java.

2. Rapid Prototyping and Development

- Enables quick implementation and testing of MapReduce logic.
- Ideal for experimenting with algorithms or processing logic without the overhead of compiling Java programs.

Hadoop Streaming

4. Integration with Existing Scripts and Tools

- Existing command-line scripts or tools can be repurposed as mappers or reducers, allowing seamless integration with Hadoop.
- Reduces development time and leverages prior knowledge and software assets.

5. Standard Input/Output Based

- Communication through stdin and stdout simplifies the data exchange between Hadoop and user scripts.
- Provides a consistent interface across different programming languages.

Working Principle of Hadoop Streaming

Hadoop Streaming follows the standard MapReduce workflow:

- **Mapper:** Reads input from STDIN, processes it, and emits key-value pairs.
- **Shuffle and Sort:** Hadoop automatically groups and sorts key-value pairs by key.
- **Reducer:** Aggregates values for each key and writes the final output to STDOUT.

Hadoop Streaming

Example: Word Count Using Python

Mapper (mapper.py)

```
python

import sys

for line in sys.stdin:
    for word in line.split():
        print(f"{word}\t1")
```

Reducer (reducer.py)

```
import sys

current = None
count = 0

for line in sys.stdin:
    word, val = line.split('\t')
    if word == current:
        count += int(val)
    else:
        if current:
            print(f"{current}\t{count}")
        current = word
        count = int(val)

print(f"{current}\t{count}")
```

Hadoop Streaming

Advantages of Hadoop Streaming

- **Language Flexibility:** Supports multiple programming languages such as Python, Bash, Perl, Ruby, PHP, and R, allowing developers and students to implement MapReduce without needing Java knowledge.
- **Rapid Prototyping and Experimentation:** Enables quick implementation and testing of MapReduce logic, reducing development time and facilitating experimentation with different algorithms or data processing techniques.
- **Simplified Learning:** Helps students and beginners understand core MapReduce concepts such as mapping, shuffling, sorting, and reducing, without dealing with the complexities of Hadoop's Java API.
- **Integration with Existing Scripts and Tools:** Existing command-line scripts or programs can be directly used as mappers or reducers, promoting code reuse and faster development.
- **Scalability and Fault Tolerance:** Fully leverages Hadoop's distributed architecture for parallel execution, automatic task scheduling, and fault-tolerant computation, ensuring reliable processing of large datasets.

Hadoop Ecosystem

The Hadoop ecosystem is a comprehensive collection of open-source tools and frameworks designed to store, manage, and analyze large volumes of data within a distributed computing environment. At its core, HDFS (Hadoop Distributed File System) provides reliable and scalable storage, while MapReduce enables the parallel processing of data across multiple nodes.

In addition to these foundational components, the ecosystem includes specialized tools that support various data processing and analytics tasks:

- **Hive:** Facilitates data querying using a SQL-like language.
- **Pig:** Allows data processing and analysis through script-based workflows.
- **HBase:** Provides real-time storage and access for structured data.
- **Sqoop & Flume:** Enable data ingestion and transfer between Hadoop and external systems.
- **Spark:** Supports high-speed, in-memory analytics for large datasets.

Together, these components form an integrated platform that enables organizations to efficiently handle diverse big data workloads, from batch processing to real-time analytics.

Hadoop Ecosystem

- **Key Tools**

1. **Apache Hive:** Provides SQL-like querying capabilities for data analysis on HDFS.
2. **Apache Pig:** A high-level platform for data processing using the Pig Latin language.
3. **Apache HBase:** A NoSQL database for real-time read/write access to large datasets on HDFS.
4. **Apache Spark:** A fast processing engine that supports in-memory computations for better performance.
5. **Apache Flume:** A service for collecting and moving large amounts of log data to HDFS.
6. **Apache Sqoop:** Transfers data between Hadoop and relational databases efficiently.
7. **Apache Zookeeper:** Manages configuration and coordination for distributed applications.
8. **Apache Oozie:** A workflow scheduler for managing Hadoop jobs.
9. **Apache Kafka:** A streaming platform for building real-time data pipelines.

Hadoop Distributed File System (HDFS)

The **Hadoop Distributed File System (HDFS)** is a highly reliable and fault-tolerant distributed file system designed specifically to store and process very large files across a cluster of commodity hardware. Unlike traditional file systems, HDFS is optimized for:

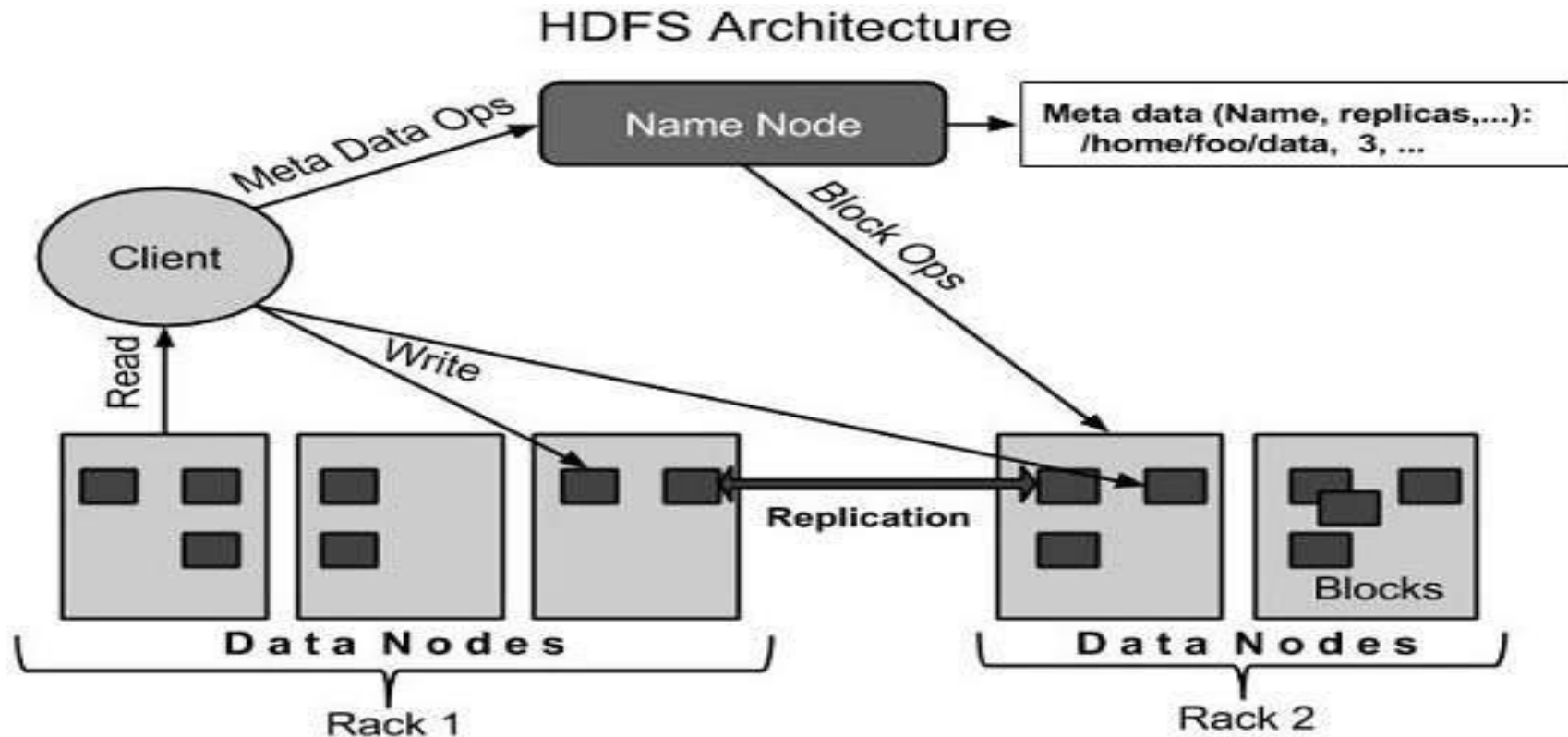
- **Streaming large files:** It is designed for high-throughput access rather than low-latency access. Applications typically read and write large datasets in a sequential manner.
- **Handling hardware failures:** HDFS assumes hardware failures are normal. It automatically replicates data blocks across multiple nodes to ensure reliability.
- **Write-once, read-many pattern:** Files in HDFS are generally written once and read multiple times, simplifying data coherency and replication.

HDFS provides a **scalable, fault-tolerant**, and **high-throughput** platform for data-intensive applications like Hadoop MapReduce.

HDFS follows a **master-slave architecture**, where the **NameNode** acts as the master managing file system metadata and directory structures, while **DataNodes** act as slaves storing the actual data blocks.

HDFS Architecture

The Hadoop Distributed File System (HDFS) is designed using a master-slave architecture, which separates the management of metadata from the storage of actual data. This architecture provides scalability, fault tolerance, and efficient handling of large datasets.



HDFS Architecture

Key Components of HDFS

1. NameNode

The NameNode is the master node of the HDFS cluster and is responsible for managing the filesystem metadata. Its key functions include:

- **Metadata Management:** Maintains the namespace of the filesystem, which includes directory structures, file names, file permissions, and the mapping of files to data blocks.
- **Namespace Operations:** Handles all client requests related to creating, deleting, renaming, or opening files.
- **Block Management:** Keeps track of the location of each data block across DataNodes and monitors replication levels to ensure fault tolerance.
- **Cluster Coordination:** Receives heartbeat signals and block reports from DataNodes to monitor their health and maintain a consistent view of the system.
- **Persistence:** Stores metadata persistently on local disk using FsImage (a snapshot of the filesystem) and EditLog (records of changes), ensuring recoverability in case of failure.

HDFS Architecture

2. DataNode

DataNodes are the worker nodes in HDFS responsible for storing the actual data blocks. They perform the following key functions:

- **Data Storage:** Stores blocks of files on local disks. Each block is typically 128 MB or 256 MB in size.
- **Data Serving:** Handles read and write requests from HDFS clients directly, enabling efficient streaming of large files.
- **Replication:** Creates additional copies of blocks as instructed by the NameNode to maintain the replication factor, ensuring reliability and fault tolerance.
- **Health Reporting:** Periodically sends heartbeats and block reports to the NameNode, reporting which blocks it stores and confirming its availability.
- **Data Integrity:** Uses checksums for each block to detect and prevent data corruption during read/write operations.

HDFS Architecture

3. Secondary NameNode

The **Secondary NameNode** is an auxiliary node that assists the primary NameNode by performing **checkpointing** of the filesystem metadata:

- Periodically downloads the **FsImage** and **EditLog** from the NameNode.
- Merges the EditLog with the FsImage to create an updated, compact snapshot of the filesystem.
- Sends the updated FsImage back to the NameNode, reducing the size of the EditLog and improving startup times.

Note: The Secondary NameNode is not a backup of the NameNode and cannot take over automatically in case of NameNode failure.

4. Clients

Clients are applications or users that interact with HDFS to read, write, or manage files. Their key interactions include:

- **File Creation/Write:** The client contacts the NameNode to allocate blocks and then writes data directly to the DataNodes.
- **File Read:** The client requests file metadata from the NameNode and retrieves data blocks directly from DataNodes.
- **File Deletion/Rename:** Sends requests to the NameNode, which updates metadata and instructs DataNodes as needed.

Block Storage in HDFS

HDFS employs a block-based storage mechanism to efficiently manage and store very large files across a distributed cluster. Instead of treating a file as a single continuous entity, HDFS divides it into fixed-size blocks, which are then distributed across multiple DataNodes. This design enables scalability, fault tolerance, and high-throughput data access.

1. Block Size

In HDFS, files are split into blocks of typically **128 MB or 256 MB**, though this value can be configured. Using large block sizes reduces the amount of metadata the NameNode must store, easing memory requirements and improving system performance. It also supports long, sequential read and write operations, which match the characteristics of big data processing.

2. Block Distribution

Once a file is divided, its blocks are distributed across different DataNodes in the cluster. The NameNode manages metadata that records which blocks belong to a file and where each block is located. During read and write operations, clients access these blocks directly from the DataNodes, allowing efficient parallel processing.

3. Block Replication

To ensure reliability, HDFS replicates each block across multiple DataNodes. The default replication factor is three, meaning three copies of each block are stored on separate nodes. If a DataNode fails, the NameNode ensures that replicas remain available and initiates re-replication if necessary, maintaining fault tolerance.

Block Storage in HDFS

4. Rack Awareness

HDFS uses a rack-aware placement policy to distribute replicas across different racks. By placing block copies on separate racks, the system safeguards data against rack-level failures and optimizes network utilization, as not all data transfers occur within the same physical segment of the cluster.

5. Benefits of Block Storage

The block-based design of HDFS offers several important advantages:

- **Fault Tolerance:** Replication protects data even if nodes or racks fail.
- **High Scalability:** Blocks can be distributed across thousands of nodes, supporting massive datasets.
- **Parallelism:** Multiple blocks can be processed simultaneously, improving computational efficiency.
- **Simplified Management:** Fixed-size blocks make file allocation and metadata handling more efficient.

HDFS File Operations

HDFS provides five main file operations: Create, Append, Read, Delete, and Rename. Each operation involves interaction between the Client, Name Node, and DataNodes.

1. Create

Goal: To create a new file in HDFS and store it reliably across the cluster.

Steps:

- The client sends a create request to the NameNode, including the file path, permissions, and replication factor.
- The NameNode checks if the file path already exists and validates the client's permissions.
- If validation succeeds, the NameNode creates metadata for the file in its namespace.
- The NameNode assigns block IDs and chooses DataNodes for storing each block according to the replication factor.
- The client writes data sequentially to the assigned DataNodes.
- Data is replicated across multiple DataNodes, and the NameNode updates the metadata with block locations.

2. Append

Goal: To add new data at the end of an existing file in HDFS.

Steps:

- The client sends an append request to the NameNode with the file path.
- The NameNode verifies that the file exists and provides the locations of the last block.
- The client writes new data sequentially to the last block.
- If the last block is full, a new block is allocated by the NameNode.
- The new data is replicated across DataNodes to maintain consistency.

HDFS File Operations

3. Read

Goal: To retrieve data stored in an HDFS file efficiently and reliably.

Steps:

- The client requests the NameNode for the block locations of the file.
- The NameNode returns the addresses of the DataNodes that store the blocks.
- The client reads blocks sequentially from the DataNodes.
- Checksums are verified to ensure data integrity.
- If a DataNode fails, the client reads the required block from another replica.

4. Delete

Goal: To remove a file or directory from HDFS and free storage space.

Steps:

- The client sends a delete request to the NameNode, specifying the file or directory path.
- The NameNode removes the metadata for the file or directory from its namespace.
- If a directory is deleted, all nested files and subdirectories are removed recursively.
- The actual data blocks stored on the DataNodes are deleted asynchronously.

5. Rename

Goal: To change the name or location of a file or directory in HDFS.

Steps:

- The client sends a rename request to the NameNode with the old path and the new path.
- The NameNode updates the file system metadata to reflect the new name or path.
- The physical blocks on DataNodes remain unchanged.
- Since renaming only affects metadata, the operation completes quickly.

The Design of HDFS (Hadoop Distributed File System)

HDFS is a distributed file system designed to store and manage large datasets across multiple machines. Here's an overview of its architecture, features, and design principles.

Key Design Principles

1. Scalability (Horizontal Scaling)

Hadoop is designed to scale out rather than scale up. Additional storage and computational capacity can be achieved by adding inexpensive nodes to the cluster, rather than relying on costly high-end servers. This design principle allows Hadoop to handle petabytes of data efficiently across large-scale distributed systems.

2. Fault Tolerance

Hadoop anticipates hardware failures as a normal occurrence and provides mechanisms to tolerate them. Data blocks are replicated across multiple nodes (typically three copies in HDFS) to prevent data loss. Additionally, computation tasks are automatically rerun on other nodes if failures occur, ensuring reliable data processing without manual intervention.

3. Data Locality

Hadoop minimizes data movement by executing computations close to the location of data. This principle reduces network congestion and enhances processing speed, which is especially important for large datasets.

The Design of HDFS (Hadoop Distributed File System)

4. High Throughput over Low Latency

Hadoop is optimized for batch processing and high-throughput access rather than low-latency operations. It is designed to process large datasets in parallel efficiently, making it suitable for analytical workloads rather than real-time applications.

5. Simplicity and Cost-Effectiveness

Hadoop emphasizes the use of simple abstractions, such as files and key-value pairs, on clusters of low-cost hardware. This approach reduces operational costs while maintaining high reliability and performance.

6. Streaming Data Processing (Write Once, Read Many)

HDFS follows a write-once, read-many access model. It is optimized for sequential reads and writes rather than random access, which aligns with the requirements of most big data analytics applications.

7. Flexibility and Extensibility

Hadoop supports a wide range of data types, including structured, semi-structured, and unstructured data. It integrates seamlessly with tools such as Hive, Pig, Spark, and HBase, enabling diverse data processing workflows and extending the framework's functionality.

Hadoop Command Line Interface

The Hadoop Command Line Interface (CLI) is the primary mechanism to interact with the Hadoop Distributed File System (HDFS). It provides a set of commands for users and administrators to manage files, directories, and perform operations on HDFS without requiring programming. The CLI acts as the interface between the user and the Hadoop system, enabling effective data manipulation and cluster management.

Purpose of Hadoop CLI

The CLI allows users to:

- Manage files and directories in HDFS.
- Access and retrieve data stored in HDFS.
- Monitor the Hadoop cluster for health and resource usage.
- Perform administrative operations like changing file permissions and setting replication factors.
- Facilitate data movement between local file systems and HDFS.

This command-based interaction is essential because HDFS is distributed across multiple nodes, and CLI commands handle communication with the **NameNode** and **DataNodes** to locate and manage data blocks.

The general syntax for Hadoop CLI commands is:

Where:

```
hdfs fs [options] <command> <parameters>
```

- `hdfs fs` is the standard interface to interact with HDFS.
- `<command>` specifies the operation to be performed (e.g., copy, list, delete).
- `<parameters>` are file paths or other arguments required for the operation.

Hadoop Command Line Interface

- The **Hadoop Command Line Interface (CLI)** allows users to interact with the **Hadoop Distributed File System (HDFS)** and other Hadoop components (like YARN) via terminal or shell commands.
- Here's a **quick reference** of commonly used **Hadoop CLI commands**:

Purpose	Command
Show help	<code>hadoop fs -help</code>
List directory	<code>hadoop fs -ls /path</code>
Create directory	<code>hadoop fs -mkdir /dirname</code>
Upload file to HDFS	<code>hadoop fs -put localfile /hdfs/dir</code>
Download file from HDFS	<code>hadoop fs -get /hdfs/file localdir/</code>
View contents of file	<code>hadoop fs -cat /hdfs/file</code>
Remove file	<code>hadoop fs -rm /hdfs/file</code>
Remove directory	<code>hadoop fs -rm -r /hdfs/dir</code>
Copy file	<code>hadoop fs -cp /source /destination</code>
Move file	<code>hadoop fs -mv /source /destination</code>
Display disk usage	<code>hadoop fs -du -h /path</code>
Check file existence	<code>hadoop fs -test -e /path</code>

Hadoop file system interfaces

- Hadoop provides several file system interfaces to store and manage data.
- These interfaces abstract the underlying file systems and allow Hadoop to work with different types of storage systems.
- The primary Hadoop File System interface is HDFS (Hadoop Distributed File System), but Hadoop also supports other file systems through its Filesystem API.
- Hadoop provides a variety of interfaces to access and manipulate the Hadoop Distributed File System (HDFS). These interfaces operate at different abstraction levels to support end-users, application developers, administrators, and external systems. They ensure flexibility, interoperability, and ease of integration across diverse programming environments.

1. HDFS (Hadoop Distributed File System)

- The default file system used by Hadoop.
- Designed for high-throughput access to large datasets.
- Stores files across multiple machines with replication for fault tolerance.
- Supports write-once-read-many access pattern.

Hadoop file system interfaces

2. Local File System

- Refers to the native file system of the operating system (e.g., ext4, NTFS).
- Used mostly for development, testing, or small-scale applications.
- URI: [file:///](#)

3. S3 File System

- Allows Hadoop to read and write data from Amazon S3.
- URI: s3a://, s3n://, or s3://
- s3a:// is the preferred scheme due to performance and security improvements.

4. HFTP File System

- HTTP-based file system used for data transfer across HDFS clusters.
- Mainly used for read access.
- URI: hftp://

Hadoop file system interfaces

5. WebHDFS

- Provides HTTP REST API to access HDFS data.
- Useful for integration with non-Java applications.
- URI: webhdfs://

6. ViewFs (Virtual File System)

- Provides a unified namespace by allowing multiple file systems (like HDFS, S3, etc.) to be mounted at different locations.
- URI: viewfs://

7. FTP File System

- Enables reading from and writing to FTP servers.
- URI: ftp://

8. Alluxio (formerly Tachyon)

- A memory-speed virtual distributed storage system.
- Can be used as a caching layer between compute frameworks (like Spark) and storage systems (like HDFS, S3).

Hadoop: Data flow

The Hadoop data flow represents the sequence of steps through which data is collected, stored, processed, analyzed, and finally visualized within the Hadoop ecosystem. This structured flow ensures that large-scale data is efficiently managed and transformed into useful insights.

1. Data Ingestion

Data ingestion is the entry point of data into the Hadoop environment. Hadoop receives data from numerous sources such as web servers, databases, sensors, social media, and enterprise applications.

Common ingestion tools include:

- **Apache Flume** for streaming log and event data
- **Apache Sqoop** for transferring large datasets between relational databases and HDFS
- **Direct HDFS uploads** through CLI or APIs

The objective of this stage is to reliably and efficiently bring raw data from external systems into Hadoop for further operations.

2. Storage in HDFS

Once ingested, data is stored in the Hadoop Distributed File System (HDFS), which serves as Hadoop's core storage layer. HDFS breaks files into large blocks (typically 128 MB) and distributes these blocks across multiple DataNodes in the cluster. This design allows Hadoop to store massive datasets reliably and to scale horizontally as more nodes are added.

Important characteristics:

- **Replication** of blocks ensures fault tolerance
- **Distributed placement** supports parallel access
- **Centralized metadata management** by the NameNode

Hadoop: Data flow

3. Data Processing

After storage, data is processed using Hadoop's distributed computation frameworks. These tools allow tasks to run in parallel across the cluster, reducing processing time for large datasets.

Major processing tools include:

- **MapReduce** for batch and large-scale processing
- **Apache Spark** for high-speed, in-memory computation
- **Hive** and **Pig** for high-level transformation and analysis tasks

At this stage, raw data is cleaned, transformed, aggregated, or modeled depending on application requirements.

4. Data Analysis

After the data has undergone initial processing, it enters the analysis phase, where the objective is to derive meaningful insights from the processed output. Analytical tools such as Apache Pig and Apache Hive are commonly employed in this stage because they provide higher-level query languages that simplify interaction with large datasets. Using languages like Pig Latin and HiveQL, users can perform operations such as filtering, aggregation, joining, and summarization without writing complex low-level code. This analytical layer enables efficient exploration of trends, patterns, and relationships within the data, thereby supporting informed interpretation and decision-making.

Hadoop: Data flow

5. Output and Export

The results generated from analysis can be stored or transferred to various destinations based on organizational needs.

Common outputs include:

- Writing results back to HDFS
- Exporting to relational databases via Sqoop
- Storing in NoSQL systems like HBase
- Delivering to external applications or reporting platforms

This ensures that processed data can be reused, shared, or integrated with other business systems.

6. Visualization

The final stage is visualization, where analytical results are converted into graphical formats for interpretation. Visualization tools like Apache Zeppelin or Tableau help decision-makers understand complex data through dashboards, charts, and reports.

Data Ingest with Flume and Sqoop

Data Ingestion with Flume :

Apache Flume is a distributed, reliable, and scalable service designed to efficiently collect, aggregate, and transport large volumes of streaming data from multiple sources to storage systems such as HDFS (Hadoop Distributed File System). It is particularly well-suited for handling real-time log data, event streams, and sensor-generated data in big data environments.

Components of Flume:

1.1 Source

The source is the entry point of data into the Flume system. It receives or monitors data from external systems and converts it into Flume events.

Common Examples of Sources:

- Log files generated by applications or servers
- Database updates
- IoT sensors or real-time event streams

1.2 Channel

The channel acts as a temporary buffer between the source and the sink, ensuring smooth and reliable data flow. It temporarily stores events until the sink is ready to process them, preventing data loss when the sink processes data more slowly than the source or when there are sudden spikes in incoming data.

1.3 Sink

The sink is the endpoint of the Flume pipeline. It is responsible for delivering events from the channel to the target application or storage system such as HDFS or HBase, ensuring that data is reliably stored for analysis or future use.

Data Ingest with Flume

Data Flow in Flume

1. **Data Collection (Source):** Flume monitors external data sources in real-time and collects new events as they occur.
2. **Event Buffering (Channel):** Collected events are temporarily stored in channels to handle variability in data rates or temporary sink unavailability.
3. **Data Delivery (Sink):** Events are pulled from channels and written to the configured storage system, such as HDFS.

Flume Configuration

Flume is configured using a properties file, which specifies the sources, channels, sinks, and their interconnections. For example:

```
agent.sources = source1
agent.sinks = sink1
agent.channels = channel1

agent.sources.source1.type = exec
agent.sources.source1.command = tail -F /var/log/myapp.log

agent.sinks.sink1.type = hdfs
agent.sinks.sink1.hdfs.path = hdfs://namenode:9000/flume/events

agent.channels.channel1.type = memory
agent.sources.source1.channels = channel1
agent.sinks.sink1.channel = channel1
```

Use Cases of flume:

1. log collection
2. Social media and streaming data capture
3. IoT sensor data ingestion
4. Event-based metrics collection

Data Ingest with Sqoop

Apache Sqoop is a powerful, open-source tool designed for efficiently transferring bulk data between relational databases and Hadoop ecosystems. Sqoop enables seamless import of structured data from RDBMS (Relational Database Management Systems) such as MySQL, Oracle, or PostgreSQL into HDFS, Hive, or HBase, and it can also export processed data back to relational databases. It is widely used in big data environments for batch-oriented data ingestion and integration.

Components of Sqoop

1.1 Connectors

- Enable communication between Hadoop and RDBMS.
- Examples: JDBC Connector for relational databases, Hive Connector, HBase Connector.

1.2 Import

- Transfers data from RDBMS to Hadoop.
- Data can be stored in HDFS, Hive tables, or HBase.
- Supports full-table import or query-based import with parallelism for efficiency.

1.3 Export

- Transfers processed data from Hadoop back to RDBMS.
- Ensures data consistency and supports bulk writes.

Data Ingest with Sqoop

2. Data Flow in Hadoop

Import (RDBMS → Hadoop):

- Sqoop reads data from the relational database using JDBC.
- Data is transferred in parallel to Hadoop.
- Stored in HDFS, Hive, or HBase for processing.

Export (Hadoop → RDBMS):

- Reads processed files from HDFS or Hive.
- Converts data into database-compatible format.
- Writes data back to RDBMS reliably.

Command Example:

To import a table from MySQL into HDFS:

```
sqoop import \  
  --connect jdbc:mysql://localhost/mydb \  
  --username user \  
  --password pass \  
  --table mytable \  
  --target-dir /user/hadoop/mytable_data
```

Use Cases in Hadoop

1. **Importing data from databases to HDFS** for storage and processing.
2. **Loading data into Hive** to run SQL-style queries on Hadoop.
3. **Populating HBase tables** for fast, real-time access.

Hadoop Archives (HAR)

Hadoop Archives (HAR) is a specialized file archiving mechanism in Hadoop designed to reduce the large number of small files stored in HDFS. Since HDFS is optimized for handling large files, too many small files create unnecessary metadata load on the NameNode. HAR solves this problem by combining many small files into a single archive while still preserving their directory structure and allowing users to access them normally.

Hadoop Archives (HAR) is a mechanism to store a large number of small files efficiently in HDFS. HAR files reduce the Namenode memory overhead and improve the performance of HDFS.

Features of Hadoop Archives

- **Reduces NameNode Metadata Load:** Combines thousands of small files into a single archive, lowering the number of file entries stored in NameNode memory.
- **Preserves Directory Structure:** Maintains the original folder hierarchy of files inside the archive.
- **Supports Random Access:** Files inside a .har archive can still be accessed individually using har:// paths.
- **HDFS-Compatible Format:** HAR files are stored and managed like normal HDFS files, ensuring easy integration with Hadoop tools.
- **Efficient Long-Term Storage:** Suitable for archiving historical or rarely accessed data without affecting performance.
- **Index-Based Access:** Internal index files allow quick lookup and retrieval of archived files.

Hadoop Archives (HAR)

Example

Create a HAR file:

```
hadoop archive -archiveName logs.har /input/logs /archive/
```

This combines all files in /input/logs into logs.har stored in /archive/.

Hadoop I/O: Compression, Serialization Avro and File-Based Data structures.

Compression:

Compression is an essential technique used in the Hadoop ecosystem to optimize storage utilization, enhance data processing performance, and improve overall system efficiency. Because Hadoop typically handles extremely large datasets, reducing the physical size of data can significantly improve throughput and reduce operational overheads across distributed clusters.

Why Compression Is Needed?

In Hadoop-based systems, data is stored and processed across multiple nodes in a distributed manner. As datasets grow in size, the storage and I/O requirements increase proportionally. Compression becomes necessary for the following reasons:

a. Storage Efficiency

Large datasets consume considerable HDFS storage space. Compressing files reduces their size, allowing organizations to store more data within the same cluster capacity.

b. Improved I/O Performance

Data transfer speed both across the network and between disk and memory is often a bottleneck in distributed systems. Smaller compressed files require less time to read, write, and move across Hadoop nodes, thereby improving I/O efficiency.

Compression

c. Faster Data Processing

MapReduce jobs can process compressed data more quickly because less data is read from disk and transferred across the network during the shuffle and sort phases.

d. Reduced Network Bandwidth Usage

During the MapReduce shuffle phase, intermediate data is transferred from map tasks to reduce tasks. Compression minimizes network congestion by reducing the volume of data sent.

Drawbacks of Compression

a. Additional CPU Overhead

Compressing and decompressing data requires CPU power. Heavy compression algorithms may increase processing time, especially on CPU-constrained nodes.

b. Reduced Parallel Processing Capability

In compressed formats that are not splittable, Hadoop cannot process blocks in parallel. This leads to performance limitations for large file processing.

c. Compatibility Considerations

Not all Hadoop tools and file formats support every compression codec(Compression/Decompression Algorithm), requiring thoughtful selection based on use-case requirements.

Compression

Data Compression Methods/Technique(codsec) in Hadoop:

a. Gzip

- Uses the DEFLATE compression algorithm.
- High compression ratio but **not splittable**, limiting parallel processing.
- Best suited for compressing final output where high compression is needed.

b. Bzip2

- Higher compression ratio than Gzip.
- **Splittable**, allowing efficient processing of large files.
- Slower compression and decompression, preferred when space savings is important than speed.

c. Snappy

- Designed by Google for very fast compression and decompression.
- Lower compression ratio.
- Ideal for intermediate MapReduce data and real-time applications where speed is crucial.

d. LZO

- High-speed codec similar to Snappy with moderate compression.
- Can become **splittable** using indexing.
- Widely used in production for logs and large datasets due to its speed and flexibility.

Compression

Splittable vs. Non-Splittable Compression

a. Splittable Compression

Splittable formats allow Hadoop to divide a compressed file into multiple chunks (splits). These splits can then be processed in parallel by different mapper tasks.

Examples: Bzip2, LZO (with indexing)

Importance: Supports Hadoop's parallel processing model, essential for large-scale MapReduce jobs.

b. Non-Splittable Compression

Non-splittable files cannot be split for parallel processing. Hadoop must assign the entire file to a single mapper, limiting performance.

Examples: Gzip, Snappy

Impact: Suitable for small-to-medium files or for compressing final MapReduce output.

Serialization in Hadoop

Serialization is the process of converting complex data structures such as objects, records, and in-memory data into a byte stream that can be stored, transmitted, or processed across distributed systems. In Hadoop, serialization ensures that data can be efficiently moved between nodes, written to HDFS, and used by various components of the ecosystem. Hadoop commonly uses formats like Avro, which are optimized for high-volume, distributed data processing.

Need for Serialization in Hadoop

1. Data Transfer Across Nodes

Hadoop runs in a distributed environment where data frequently moves between machines. Serialization ensures that data can be transmitted in a standard, compact byte format.

2. Efficient Storage in HDFS

Serialized data consumes less space and can be stored efficiently in HDFS, improving read/write performance.

3. Inter-Process Communication

MapReduce tasks exchange key-value pairs between mappers and reducers. Serialization provides a consistent format for this communication.

4. Performance Optimization

Hadoop requires fast processing of massive datasets. Serialized formats are lightweight and reduce overhead, leading to faster computation.

5. Language and Platform Independence

Serialization allows Hadoop components written in different languages and running on different platforms to exchange data smoothly.

Avro in Hadoop

Avro is a data serialization framework in Hadoop that enables efficient, compact, and language-independent storage and communication of data. It is widely used in MapReduce workflows, Hive, Pig, and other Hadoop ecosystem tools for fast data exchange and processing.

Key Features and Concepts

1. Lightweight and Efficient

Avro stores data in a binary format, reducing storage space and improving read/write performance.

2. Schema-Based Serialization

Data is defined using a JSON schema, which ensures consistent reading and writing across applications.

3. Cross-Language Compatibility

Avro supports multiple programming languages, allowing seamless data exchange in heterogeneous Hadoop clusters.

4. Schema and Data Together

Each Avro file contains both the schema and the data, simplifying data processing and ensuring self-descriptive files.

File-Based Data Structures in Hadoop

In Hadoop, file-based data structures are specialized formats used to store, manage, and process large volumes of data efficiently. Since Hadoop is designed for distributed storage and parallel processing, these data structures allow data to be serialized, compressed, and split across multiple nodes for faster computation.

File-based data structures form a core part of Hadoop I/O, providing mechanisms to read and write data efficiently in MapReduce jobs and other Hadoop ecosystem tools.

Characteristics

- **Distributed Storage:** Data is split into blocks and stored across multiple nodes in HDFS.
- **Splittable:** Many formats allow parallel processing of different file sections.
- **Compressed:** Most formats support compression to reduce storage and network usage.
- **Schema Support:** Formats like Avro include schema information, ensuring consistency when reading and writing data.

File-Based Data Structures in Hadoop

Classification of File-Based Data Structures

1 Text Files

- **Description:** Plain text files in formats such as CSV or TSV.
- **Characteristics:** Human-readable, simple to generate and interpret.
- **Limitations:** Lack schema definition; inefficient for complex queries and large-scale processing due to storage and I/O overhead.
- **Use Case:** Log files, simple data exchanges, and small datasets.

2 SequenceFile

- **Description:** A binary key-value pair format designed for efficient processing in Hadoop.
- **Characteristics:** Supports splitting and compression; optimized for MapReduce intermediate data.
- **Limitations:** Not human-readable.
- **Use Case:** Intermediate storage of MapReduce outputs, bulk data transport.

3 Avro

- **Description:** schema-defined serialization framework.
- **Characteristics:** Self-describing (schema stored with data), compact, supports compression, and interoperable across multiple programming languages.
- **Limitations:** Requires schema management; slightly more complex compared to plain text.
- **Use Case:** Data serialization, long-term storage, RPC communication.

File-Based Data Structures in Hadoop

4 ORC (Optimized Row Columnar)

- **Description:** A highly efficient columnar format, optimized for Hive.
- **Characteristics:** Provides lightweight compression, indexing, and high-speed read/write operations.
- **Limitations:** Primarily used within Hive environments.
- **Use Case:** Large Hive tables, online analytical processing (OLAP).

Importance in Hadoop I/O

1. **Efficient Data Storage:** Optimized formats reduce storage space and improve access speed.
2. **Parallel Processing:** Splittable formats enable distributed computation across Hadoop clusters.
3. **Schema Evolution Support:** Formats like Avro allow modifications to data schema without breaking compatibility.
4. **I/O Optimization:** Columnar formats enhance read performance for analytical queries, reducing unnecessary disk I/O.

Thank You !